COSC 42: Distributed Computing

Problem 1. FCFS Mutex with Fetch-and-Add and Bounded Values.
Recall that the Ticket Lock from lecture requires a counter whose value grows unboundedly. The goal in this problem will be to design a lock with fetch-and-add that ensures that values held in memory are bounded to polynomials in n, and thereby only require $O(\log n)$ bits to store.
Design an n-process mutual exclusion lock that satisfies Mutex, Starvation Freedom, FCFS, and Bounded Exit. You may only use the read, write, and fetch-and-add shared-memory instructions in your algorithm. The algorithm must not require shared or local variables with more than $O(\log n)$ bits. You may assume that n is fixed and known, and that the processes are named $1, 2, \ldots, n$.

Solution.

```
Shared Variables:
    generation ∈ {0,1}, initialized to 0
    choosing[π] = False, for all π ∈ Π
    tickets[π] = (0,0), for all π ∈ Π // Each ticket is (generation, ticket number)
Local Variables:
    t, initialized arbitrarily
Try(π):
Doorway:
    T1 : choosing[π] = True
    T2 : tickets[π][0] = generation
    T3 : t = 0
    T4 : for π' ∈ Π:
            if tickets[π'][0] = tickets[π][0]:
               t = max(t, tickets[π'][1])
    T5 : tickets[π][1] = t + 1
    T6 : choosing[π] = False

Waiting Room:
    T7 : for π' ∈ Π, π' ≠ π:
            await not choosing[π']
            if tickets[π'][0] = tickets[π][0]:
                await (tickets[π'][1] == 0
                or (tickets[π][1], π) < (tickets[π'][1], π')
                or tickets[π'][0] ≠ tickets[π][0])
            else:
                await (tickets[π'][1] == 0
                or tickets[π][0] ≠ generation
                or tickets[π'][0] == tickets[π][0])
```

```
Exit(π):
    E_1 : generation = 1 − tickets[π][0]
    E_2 : tickets[π][1] = 0
```

We will now prove that the algorithm satisfies Mutex, Starvation Freedom, Live-Lock Freedom, FCFS, Bounded Exit, and only takes $O(\log n)$ space.

Note that at most n processes share the same generation at a given time. Let the shared generation be g. Any process waiting with a ticket from generation $1-g$ has priority over processes holding generation-g tickets. In particular, after a process with generation $1-g$ exits, it sets the shared generation to g, preserving the priority of other outstanding generation-$(1-g)$ tickets. The shared generation cannot switch back while those earlier tickets are still waiting.

- **Mutex:** Suppose for contradiction that two processes $\pi, \pi' \in \Pi$ are in CS at the same time. There are two cases:

  1. **Same Generation:** Note that ticket numbers are assigned in a non decreasing order in the same generation, and we break ties by minimality of ticket number and process id. WLOG assume that π has the smaller pair. Then by T_7, π' must wait for π to start the Exit section before entering the CS since π's ticket remains nonzero until it exits, and since the choosing array ensures that π' waits until π assigns its ticket, both processes cannot simultaneously be in the CS.
  2. **Different Generation:** If π and π' have different generation, by the generation ordering property, tickets from the previous generation have priority over tickets from the current generation, thus the process with the new generation must wait until the processes with the previous generation has exited the CS.

  Thus at most one process is in the CS at any time.

- **Bounded Exit:** The Exit section consists of two writes and hence can be completed in a bounded number of steps of the process.

- **FCFS:** Suppose π finishes its doorway before π' starts, then:

  1. π has already entered CS and FCFS is satisfied.
  2. they are in the same generation, then since π has already published its ticket, π' accounts for its ticket when computing the maximum, and thus, $tickets[\pi'][1] > tickets[\pi][1]$, so π' waits for π to reset its ticket to 0 which happens only when π exits.
  3. they are in different generations, then if π' receives the opposite generation, then by the generation ordering property the outstanding tickets from π's generation get priority. And thus, π' must wait till all processes from that generation (including π) are done.

In both cases, π' cannot enter the CS before π. Therefore, the algorithm satisfies FCFS.

- **Live-Lock Freedom:** Suppose at least one process is in the Try Section waiting to enter the CS. In a fair run, all processes currently in their doorways eventually finish, since the doorway requires only $O(n)$ steps. Any processes in CS or the Exit Section finish and exit. Since no new process enters the CS the shared generation variable eventually stops changing.

  Then of the waiting processes we can pick the one with the highest priority first by generation and then by ticket number and process ID. This process has no higher-priority process to wait for, so it eventually completes its waiting room and enters the CS, contradicting our assumption. Thus the algorithm satisfies Live-Lock Freedom.

- **Starvation Freedom:** Since the algorithm satisfies Live-Lock Freedom, Mutex and FCFS, it also satisfies Starvation Freedom.

- $O(\log n)$ **Space:**

  1. The shared generation variable has values in $\{0, 1\}$ and needs only 1 bit.
  2. Each entry in the choosing array is boolean and thus also requires 1 bit.
  3. Each entry in the tickets array consists of a (generation, ticket number) tuple, where the ticket number is bounded by n. Note that process ids are also bounded by n. Thus both ticket numbers and process ids require at most $O(\log n)$ bits.

  Thus all shared and local variables take at most $O(\log n)$ bits, and the total shared space is $O(n \log n)$ bits.

Problem 2. FCFS from Starvation Freedom .
Reduce n-process FCFS Mutex to n-process Starvation Free Mutex. That is, let $n \geq 2$ be any fixed and known integer, and let L be a solution to the mutual exclusion problem that satisfies Mutex and Starvation Freedom, construct by reduction a lock L' for n processors that satisfies Mutex, Starvation Freedom, and First-Come-First-Served (FCFS). Your reduction should only use the read and write shared-memory instructions (i.e., it should not use RMW instructions). You may assume that n is known and that the processes have names in the set $\{1, 2, \ldots, n\}$.

Solution.

Let L be a solution to the mutual exclusion problem that satisfies Mutex and Starvation Freedom for n processes. Note that then by definition, in a fair run, we are guaranteed that every process that tries to enter CS will eventually enter it. We will construct L' by reduction as follows:

```
L':
Shared Variables:
    tickets[]: initialized to tickets[π] = 0, ∀π ∈ Π
Local Variables:
    current: initialized arbitrarily
    t: initialized to 0

Try(π):
Doorway:
T1 : t = 0
T2 : for π' ∈ Π:
         t = max(tickets[π'], t)
T3 : tickets[π] = t + 1
Waiting Room:
T4 : current = (tickets[π], π)
T5 : for π' ∈ Π:
         if π'! = π and tickets[π']! = 0 and (tickets[π'], π') < current:
             current = (tickets[π'], π')
T6 : if current! = (tickets[π], π):
         go to T4
    else:
         L.Try(π)

Exit(π):
E1 : L.Exit(π)
E2 : tickets[π] = 0
```

Note that we break ticket ties by process ids. We will now prove that L' satisfies Mutex, Starvation Freedom, and FCFS.

- **Mutex:** Every process must get the L Lock before entering the CS. Since L satisfies

Mutex, so does L'.

- **FCFS:** Suppose π completes its doorway before π' starts. If π hasn't finished its passage, then $t_\pi < t_{\pi'}$. Therefore, π' cannot pass T6 until π resets its ticket to 0, which happens after π exits the CS. If their doorways overlap, FCFS imposes no restriction on their relative order of accessing the CS.

- **Starvation Freedom:** Once π obtains a ticket, no process beginning its doorway afterward can obtain a smaller ticket while π is waiting. At most $n-1$ existing attempts can have priority over π.

  Consider the process with the smallest (ticket, process id) pair among these attempts. In a fair run, it eventually completes its waiting room and enters $L.Try()$. By the Starvation Freedom of L, it eventually enters and leaves the CS, resetting its ticket to 0. Repeating this argument, all processes with a smaller (ticket, process id) pair than π eventually finish. Thus, π eventually enters $L.Try()$, and by the Starvation Freedom of L, eventually enters the CS.

  Also note that the doorway ($T_1 - T_3$) can be completed in a bounded number of the process' steps and hence Starvation Freedom is satisfied.

Problem 3. $O(1)$ **DSM RMR Mutex with Fetch-and-Add (Optional)**.
Read about the MCS Lock that uses the fetch-and-store instruction to achieve a lock satisfying FCFS with $O(1)$ RMR complexity in both the DSM and CC models.
Design an n-process mutual exclusion lock that satisfies Mutex, Starvation Freedom, FCFS, and Bounded Exit. You may only use the read, write, and fetch-and-add shared-memory instructions in your algorithm. Your algorithm must have $O(1)$ RMR Complexity on both DSM and CC machines. You may assume that n is fixed and known.